Introduction

Every n8n builder hits the same wall eventually: the workflow runs perfectly, the data gets scraped, scored, transformed — and then it just stops. The last node fires once and you're left staring at a green checkmark wondering where the data actually went. That's not a logic bug. It's a missing decision: what happens the *second* time this workflow runs?

This is the part of automation that interview-style thinking actually maps onto directly. If you've ever worked with a cache, a message queue consumer, or a REST API that needs to be safe to retry, you've already solved a version of this problem. The pattern here — read existing state, check for a match, branch into create-or-update — is the exact same idea behind idempotent API design and upsert operations in real databases. Learning it in n8n with Google Sheets is a low-stakes way to build an instinct you'll reuse constantly, whether you're writing to Postgres, calling a CRM API, or syncing two systems that both think they own the data.

The reason this trips up almost everyone starting out isn't that the logic is hard. It's that "just append a row" feels done. It runs, it writes, it looks correct — right up until you run the workflow twice and your sheet has two identical leads sitting in it, each with their own ID, email, and score. Nothing crashed. Nothing errored. You just quietly started lying to yourself about how many leads you have.

Problem Overview

You have a workflow that produces good data — in this case, scraped and scored leads — and a destination that's supposed to store it: a Google Sheet, or an Airtable base. Think of it less as "where do I put this spreadsheet row" and more as "how do I write to a data store without corrupting it on a second run."

The constraint that makes this interesting: the destination has no built-in uniqueness enforcement. Unlike a SQL table with a primary key constraint, a spreadsheet row doesn't reject a duplicate ID. It will happily let you write the same lead in ten times if you ask it to. So the responsibility for not duplicating moves entirely onto your workflow logic.

The goal is a workflow that, given a lead: 1. Checks whether that lead already exists in the destination. 2. If it does, updates the existing row in place. 3. If it doesn't, creates a new row.

This needs to hold true no matter how many times the workflow runs, whether it's triggered manually, on a schedule, or re-run after a partial failure.

Example

Say your scraper produces this lead:

```
id: 1042
name: Ana Ortiz
email: Ana@x.com
score: 87
```

Run 1: The sheet is empty. n8n reads all rows, finds nothing with `id = 1042`, so it appends a new row. Sheet now has one row for Ana.

Run 2 (same workflow, same lead, run again by accident or by schedule): n8n reads all rows again. This time it finds a row where `id = 1042`. Instead of appending, it updates that row — maybe the score changed to 91 after a re-scrape. The sheet still has exactly one row for Ana, now with the fresh score.

What would have happened with blind-append: Run 1 produces one row. Run 2 produces a second row with the same ID. Now any downstream automation that counts leads, emails leads, or scores leads by ID is working against duplicated, inconsistent data. The bug doesn't show up in this run — it shows up three weeks later when someone asks why Ana got two follow-up emails.

Intuition

Here's how an experienced engineer actually thinks through this before touching a single node.

Start from the mental model of what a spreadsheet *is*: rows with columns. A Google Sheet is rows with columns. A database table is rows with columns. For small, low-write-volume data where a human wants to open the thing and eyeball it, a spreadsheet isn't a weaker version of a database — it's the right-sized tool. You don't need SQL or a hosted Postgres instance to track two hundred leads.

But the moment you treat rows-with-columns as a database, you inherit database problems, and the first one is always the same: how do you avoid inserting the same logical record twice? Every real database answers this with a primary key constraint or an upsert operation. A spreadsheet has no such constraint built in — so you have to build the check yourself, by hand, in the workflow.

The naive move is reaching for an "Update" node and assuming it'll just work the first time. It won't — update operations update rows that exist; they don't create ones that don't. If the lead has never been written before, an Update node has nothing to match against and either errors or silently does nothing. That failure mode is sneaky precisely because it's quiet. You don't get an obvious crash — you get a workflow that "ran successfully" while doing nothing useful.

So the actual insight is: **read before you write, and branch on what you find.** Pull every existing row into the workflow, compare the incoming record against them on some stable identifier, and let that comparison decide which of two paths — update or create — the record takes.

The identifier choice matters more than it looks. A plain equality check is case-sensitive. If a lead comes in once as `Ana@x.com` and again as `ana@x.com`, a straight string match treats those as different values and you're back to silent duplicates — just via a different mechanism than before. That's why you match on the ID column, not the email. IDs don't have a casing problem. If you ever do have to match on a text field like email, normalize both sides to lowercase before comparing.

Brute Force Solution

The "brute force" version of this problem is the blind-append approach most people start with:

Idea: Every time the workflow runs, append the incoming data as a new row. No read, no comparison, no branching.

Algorithm: 1. Receive the record. 2. Append it to the sheet/table.

Advantages: - Trivial to build — one node, no logic. - Fast, since there's no read step before the write.

Disadvantages: - Produces a duplicate row every time the same record flows through more than once. - No way to update existing data — a lead's score changing never gets reflected, because nothing ever overwrites anything. - Silently corrupts the dataset over time; the failure isn't visible until someone audits the data.

Complexity: O(1) time and space per run, since there's no read — but this is cheap specifically because it's incomplete. It's solving a smaller, wrong problem.

Optimal Solution

The fix adds exactly one read and one branch:

```
[Trigger] → [Read all rows from Sheet/Table]
                      ↓
         [Compare incoming ID against existing IDs]
                      ↓
                ┌─────┴──────┐
         Match found    No match
              ↓               ↓
     [Update existing row] [Append new row]
```

Step by step:

1. **Connect once.** Authenticate to Google Sheets (or Airtable), point the node at the specific sheet/tab or base/table you're writing to. This is pure plumbing — no logic yet, just establishing where reads and writes go.
2. **Read before you decide anything.** A node reads every existing row back into the workflow. This gives n8n visibility into what's already there — without this step, the workflow is flying blind.
3. **Match on a stable field.** An IF node (or equivalent) compares the incoming record's ID against the IDs already in the sheet. ID is chosen deliberately over email or name because it has no casing or formatting ambiguity.
4. **Branch.**
5. Match found → route to an Update node, which writes into the *existing* row rather than creating a new one.
6. No match → route to an Append/Create node, which adds a brand-new row.
7. **Airtable is the same shape.** Instead of reading rows and scanning them manually, you search for a matching record by a field directly. But the decision tree is identical: found means update, not found means create. The node names change; the pattern doesn't.

The table below summarizes the shift:

	Blind Append	Check-Then-Branch
Reads before writing	No	Yes
Handles repeat runs	No — duplicates	Yes — idempotent
Reflects updated data	No	Yes
Extra nodes required	0	2 (read/search + IF)

Python Code

n8n is visual, but the underlying logic is a few lines of straightforward Python — useful both to understand what the nodes are doing internally and if you ever port this pattern into a script:

```
from dataclasses import dataclass


@dataclass
class Lead:
    id: str
    name: str
    email: str
    score: int
```

```
def find_matching_row(rows: list[dict], lead: Lead) -> dict | None:
    """Match on ID only — IDs have no casing ambiguity, unlike email."""
    for row in rows:
        if row["id"] == lead.id:
            return row
    return None


def upsert_lead(rows: list[dict], lead: Lead) -> list[dict]:
    existing = find_matching_row(rows, lead)
    if existing is not None:
        existing.update(name=lead.name, email=lead.email, score=lead.score)
    else:
        rows.append(
            {"id": lead.id, "name": lead.name, "email": lead.email, "score": lead.score}
        )
    return rows
```

Code Walkthrough

- `Lead` is a small dataclass standing in for whatever record your scraper/scorer produces — the equivalent of the data flowing out of your n8n trigger.
- `find_matching_row` is the read-and-compare step: it scans existing rows (the equivalent of n8n's "read all rows" node) looking for one whose `id` matches the incoming lead's `id`. This mirrors matching on ID rather than email specifically to dodge the case-sensitivity trap.
- `upsert_lead` is the branch. If `find_matching_row` returns something, we mutate that row in place (the Update path). If it returns `None`, we append a new dict to `rows` (the Create path). The `if/else` here *is* the IF node in your n8n workflow — same decision, different environment.
- Running `upsert_lead` twice with the same `lead.id` produces one row, not two — the whole point.

Dry Run

Starting state: `rows = []`

Call 1: `upsert_lead(rows, Lead(id="1042", name="Ana Ortiz", email="Ana@x.com", score=87))` - `find_matching_row` scans an empty list → returns `None`. - Falls to the `else` branch → appends `{"id": "1042", "name": "Ana Ortiz", "email": "Ana@x.com", "score": 87}`. - `rows` now has 1 entry.

Call 2 (same lead, rescraped with a new score): `upsert_lead(rows, Lead(id="1042", name="Ana Ortiz", email="Ana@x.com", score=91))` - `find_matching_row` scans `rows`, finds the dict where `id == "1042"` → returns it. - `existing.update(...)` overwrites `score` to `91` in place. - `rows` still has exactly 1 entry, now with the updated score.

Two runs, one row, correct data. That's the entire test of whether this pattern is working.

Complexity Analysis

- **Time:** O(n) per run, where n is the number of existing rows, because `find_matching_row` does a linear scan. This is correct and necessary at small scale — you genuinely have to check every existing row (or let the API do an indexed search, as Airtable does) because there's no persistent index structure backing a spreadsheet.
- **Space:** O(n) to hold all existing rows in memory during the comparison, since the whole dataset gets read in before any decision is made.
- These bounds are fine for hundreds or low thousands of rows. They stop being fine once "read everything, then search" starts costing real time on every single run — which is exactly the ceiling this pattern runs into (see Edge Cases below).

Alternative Solutions

Search-by-field instead of read-everything-then-scan: This is what Airtable's native search does — instead of pulling every record into the workflow and comparing locally, you ask Airtable's API to find a record matching a field directly. It's the same O(n) search cost in the worst case, but it's done server-side with indexing, which tends to be faster and lighter on your workflow's memory as the table grows. Prefer this whenever your destination supports a native "find by field" operation rather than a full table dump.

A real database with a unique constraint: Once you're hitting rate limits or latency from reading the whole table every run, the actual fix is moving to Postgres/MySQL/SQLite with a unique constraint on the ID column and an `INSERT ... ON CONFLICT DO UPDATE` (upsert) statement. The database enforces uniqueness for you instead of your workflow logic doing it manually — strictly better once the overhead of standing up a database is justified by write volume.

Edge Cases

- **Empty sheet/table:** `find_matching_row` correctly returns no match for everything, so the first run is all creates. Verify your workflow doesn't choke on a sheet with just a header row and

no data rows.

- **Duplicate IDs already present from a previous blind-append bug:** The matcher will find the *first* match and update it, silently leaving the duplicate rows untouched. You need a one-time cleanup pass before this pattern can trust the existing data.
- **Case-mismatched text fields used as the match key:** As covered above, matching on email without normalizing case produces silent duplicates. Always lowercase both sides, or avoid text fields as the match key entirely.
- **ID present but every other field empty:** Make sure your Update path doesn't overwrite good data with blank incoming fields — decide explicitly whether a partial record should overwrite or merge.
- **Very large sheet (thousands of rows):** The read-everything approach gets measurably slower and starts bumping into Google's API rate limits — this is the signal to move to a real database.

Common Mistakes

1. **Using an Update node alone, with no prior read/check.** It has nothing to match against on the first run and fails or silently no-ops.
2. **Matching on email without normalizing case.** `Ana@x.com` and `ana@x.com` are treated as different values, producing a duplicate that looks like a legitimate new lead.
3. **Forgetting to re-test both branches.** It's easy to test the "create" path, ship it, and never actually verify the "update" path fires correctly on a second run.
4. **Skipping the read step to save a node.** This looks like a harmless optimization but removes the entire mechanism that makes the workflow safe to re-run.
5. **Assuming Airtable and Google Sheets need different logic.** The node names differ (search-by-field vs. read-and-scan) but the branching structure — found means update, not found means create — is identical; don't over-engineer a separate design for each.
6. **Not planning for scale.** Treating "read everything, every run" as permanent instead of recognizing it as a pattern with a shelf life once row counts grow.

Interview Questions

- How would you make an API endpoint idempotent, and how does that relate to this create-or-update pattern?
- What's the difference between an upsert and separate insert/update operations, and when would you prefer one over the other?

- How would you detect and clean up duplicate records that already exist in a dataset before applying this pattern going forward?
- At what point does a linear "scan everything" matching strategy become a performance problem, and what would you replace it with?
- Why is case-insensitive comparison necessary for some fields (email) but not others (numeric or UUID-style IDs)?

Similar Problems

- **LeetCode 1: Two Sum** — same core skill of scanning a collection to find a match against a target value, just with numbers instead of records.
- **LeetCode 217: Contains Duplicate** — directly about detecting whether a value already exists in a collection, the exact check this pattern relies on.
- **LeetCode 49: Group Anagrams** — practice at matching/grouping records by a derived key, similar to deciding what field determines "the same record."
- **LeetCode 705: Design HashSet** — building the underlying matching structure that a more scalable version of this upsert pattern would use instead of a linear scan.

Key Takeaways

- Append-only workflows are a time bomb: they work until the second run, then quietly start duplicating data.
- The fix is always the same shape: read what exists, compare on a stable identifier, branch into update-or-create.
- Match on IDs, not text fields with casing ambiguity like email — or normalize case first if you must.
- This isn't a Google Sheets quirk or an Airtable quirk — it's the general rule for any workflow writing into something stateful: a CRM, a database, an API with unique records.
- The pattern has a scaling ceiling; read-everything-then-search gets slower and rate-limit-prone as the dataset grows, which is the actual signal to graduate to a real database.

Watch the Video

This entire pattern is built live, node by node, in both Google Sheets and Airtable — watching it assembled makes the branching logic much more concrete than reading about it. Check out the full build here: {LINK}

About the Series

This is part of **Fun with Learning Technology**, a series focused on actually building the automations and systems developers talk about instead of just explaining them in theory. Each lesson takes a problem that looks small — like "where does this data go" — and uses it to build durable engineering instincts that transfer well beyond the specific tool in front of you.

Call To Action

If this pattern saved you from a future duplicate-data headache, give the video a like on YouTube and subscribe for the rest of the series. For deeper written breakdowns like this one, subscribe to the Substack. Drop a comment if something felt off or you want the next lesson to cover a specific destination (Postgres, Notion, a custom API) — and share this with anyone on your team still blind-appending into a spreadsheet.

Quick Review

n8n: how to use Google Sheets as a database?

Read all rows, filter/match in a logic node, then append or update based on match result — no real DB needed for simple cases.

Why check-then-branch before writing a row?

Prevents duplicate inserts: read existing rows first, branch on whether a match exists, then choose Append vs Update.

Common bug when matching rows by a field?

Matcher does loose/partial matching (e.g. substring or type mismatch like string vs number) and falsely matches unrelated rows.

Sheets row count vs Airtable record limits — tradeoff?

Sheets is free/simple but slow at scale and matching is manual; Airtable has structured fields, views, and faster lookups but API quotas.

Where does this read-then-write pattern break?

Under concurrent writes — two workflow runs can both read 'no match' and both append, creating duplicate rows (race condition).

Interview follow-up: how to avoid the race condition above?

Add a unique-key constraint/dedup step downstream, or serialize writes (locking/queue) instead of relying on read-then-write checks.
